Using Blocks for Transactions

Although blocks are often used as the target of an iterator, they have other uses. Let's look at a few.

You can use blocks to define a chunk of code that must be run as part of some kind of transaction. For example, you'll often open a file, do something with its contents, and then need to ensure that the file is closed when you finish. Opening and closing the file is a transaction that you want to happen together regardless of what you do with the contents. Although you can manage a transaction using conventional linear code, a version using blocks is simpler (and turns out to be less error prone). A naive implementation (ignoring error handling) could look something like the following:

```
class File
  def self.open_and_process(*args)
    f = File.open(*args)
    yield f
    f.close()
  end
end

File.open_and_process("testfile", "r") do |file|
  while line = file.gets
    puts line
  end
end
```

produces:

```
This is line one
This is line two
This is line three
And so on...
```

The method open_and_process is a *class method*—its receiver is the class itself, and it may be called independently of any particular file object. We'll discuss class methods more in Chapter 5, More About Methods, on page 85. We want open_and_process to take the same arguments as the conventional File.open method, but we want to just pass them through no matter what the arguments are. So, we've specified the parameter list as *args meaning "collect the positional parameters passed to the method into an array named args". We then call File.open, passing it *args as an argument. This expands the array back into individual parameters. The net result is that open_and_process transparently passes its non-block arguments to File.open.

Once the file has been opened, open_and_process calls yield, passing the open file object to the block. When the block returns, the file is closed. In this way, the responsibility for closing an open file has been shifted from the users of file objects to the file objects themselves.

The technique of having files manage their own life cycle is so useful that the class File supplied with Ruby supports it directly. If File.open has an associated block, then that block will be invoked with a file object, and the file will be closed when the block terminates. This is interesting, because it means that File.open has two different behaviors. When called with a block, it executes the block and closes the file. When called without a block, it just returns the file object. This is made possible by the method block_given?, which returns true if a block